Prague University of Economics and Business
Faculty of Informatics and Statistics

Material modification of 3D objects on the web

BACHELOR THESIS

Study program: Multimedia in the Economic Practice

Author: Václav Dušek

Supervisor: Bc. et Bc. Jaroslav Svoboda, MSc

Prague, June 2024

Declaration

I hereby declare that I have written the bachelor's thesis solely by myself under the supervision of Bc. et Bc. Jaroslav Svoboda, MSc and that no other sources, other than listed in the references, have been used.

Václav Dušek

In Prague, 27. 6. 2024

Acknowledgements

I would like to thank Bc. et Bc. Jaroslav Svoboda, MSc for supervising this thesis and helpful comments.

The thesis was typeset in LaTeX.

Abstract

The main goal of this thesis was to solve the visible texture repetition in 3D graphics on the web. For the rendering of the 3D scene is used WebGL as the main technology for 3D graphics on the web. The WebGL is handled with the help of the Three.js framework. The texture repetition is solved by the implementation of several techniques. These techniques are distance tiling and micro and macro variation. The texture bombing was not implemented because it is not suitable for some types of textures. The material, that will not have noticeable texture repetition, was created by inheriting properties from MeshStandardMaterial, which comes with Three.js. The mentioned techniques were implemented with the use of GLSL as a part of vertex shader and fragment shader. These parts of GLSL codes were injected into the original GLSL code that was inherited from MeshStandardMaterial. The last step was to add new properties to the material to be able to control variables in the GLSL code. This results in material, that hides the texture repetition by adding more visual variation to the final look.

Keywords

3D graphic, JavaScript, GLSL, shader, WebGL, Three.js

Abstrakt

Hlavním cílem této práce bylo vyřešit viditelné opakování textury ve 3D grafice na webu. Jako hlavní technologie pro vykreslování 3D scény na webu je použito WebGL. Práce s WebGL probíhá za pomoci Three.js frameworku. Opakování textury je vyřešeno implementováním několika technik. Konkrétně jsou to vzdálenostní škálování and micro a macro variace. Technika texture bombing nebyla použita, protože pro některé textury není vhodná. Materiál, který nebude mít viditelné opakování textury, byl vytvořen tak, že zdědil vlastnosti MeshStandardMaterialu, který je součástí Three.js. Zmíněné techniky byly implementovány jako části vertex a fragment shaderů za pomocí GLSL. Tyto části kodu byly poté vloženy kódu, který byl zděděn od MeshStandardMaterialu. Posledním krokem bylo přidání nových vlastností materiálu, skrz které lze ovlivňovat proměné v GLSL kodu. Tím vzniknul materiál, který schovává opakování textury přidáním vizuální variace.

Klíčová slova

3D grafika, JavaScript, GLSL, shader, WebGL, Three.js

Contents

Introduction **9**

1 History of 3D on the web **11**

2 JavaScript **12**
2.1 Overview . . . 12
2.2 History of JavaScript . . . 12

3 JavaScript libraries for 3D **14**
3.1 Babylon.js . . . 14
3.2 Three.js . . . 14

4 Shaders and GLSL **15**
4.1 Vertex Shader . . . 15
4.2 Fragment Shader . . . 16
4.3 GLSL variable qualifiers . . . 16
4.4 GLSL preprocessor directives . . . 17

5 Noise **18**
5.1 Perlin Noise . . . 18
5.2 Simplex Noise . . . 19
5.3 Fractal Brownian Motion . . . 19

6 FLOSS - an approach to licensing **20**
6.1 Public Domain . . . 20
6.2 GNU General Public License v3 . . . 21
6.3 MIT license . . . 21
6.4 Apache license 2.0 . . . 21
6.5 Chosen license . . . 21

7 Preliminary research **22**

8 Development of material modifying library for Three.js **23**
8.1 Modification of Three.js materials . . . 23
8.2 Texture repetition (tiling) . . . 24
8.2.1 Research . . . 24
8.2.2 Implementation . . . 27

Conclusion **33**

Bibliography **34**

A Source codes of computational procedures **37**

List of Figures

5.1 Simplex noise with one, three and six octaves 19

8.1 Plane with seamless texture 25
8.2 Plane with distance tilling 28
8.3 Plane with micro variation 30
8.4 Plane with macro variation 31
8.5 Plane with all methods 32

List of abbreviations

API application programming interface
HTML Hypertext Markup Language
GLSL OpenGL Shading Language
WebGL Web Graphics Library
GPU graphics processor unit
VRML virtual reality markup language
WWW World Wide Web
X3D Extensible 3D
W3C The World Wide Web Consortium
FLOSS Free/Libre and Open Source Software
GNU GNU's Not Unix
ISO International Organization for Standardization
TC39 Technical Committee 39
ECMA European Computer Manufacturers Association

Introduction

When a user looks at large landscape planes in 3D scenes, the first that disturbs them is an obvious texture repetition. This is a very common problem and on the web, one needs more skill to adjust and solve this problem than in standalone 3D computer graphics software. For example in Blender or Unreal Engine, 3D artists can easily modify the algorithm of the landscape material in the node editor. On the web, 3D rendering is done in WebGL where the algorithms for materials are written in GLSL. This approach is more complex with a steep learning curve. A beginner needs to learn a whole new programming language and paradigms of rendering pipelines to achieve good results.

One may ask why not use the previously mentioned software to prepare the whole scene with the modified material. Unfortunately, each software handles the material differently therefore only a few common data are transferable but not algorithms. The transferred material would likely look very different from the initial ones. Even for the web environment some node editors exist but these are mostly still in development and they lack some features or the features are behind a paywall, as an example can be given uploading a texture, therefore they have limitations. One limitation, that probably will not be overcome, is the application of the created material.

The creation of a 3D scene on the web is based on code. It means basically creating a WebGL application. The output of these node editors is a piece of code that has to be copied and pasted into the application code. Any change would require deleting previously pasted code and copying and pasting the new one. In comparison with the previously mentioned standalone software which has the node editor integrated and the results are visible immediately, this is time-consuming and inconvenient. Most standard users do not have the knowledge of the material algorithms to write their own GLSL code or create their node tree. They usually just watch a YouTube tutorial which ends up in them copying random code or node trees that they are not able to adjust later. Usually, common problems like texture repetition do not have many solutions but they are addressed with one or a combination of a few techniques with a little bit of adjustments. For these reasons, it is convenient to create a code, that can be imported and would be easily adjustable through parameters.

It is worth mentioning that working with WebGL is hard, therefore several frameworks were made to make the work with WebGL easier and more accessible. These frameworks have several prebuilt materials for general purposes but they do not solve the mentioned problem. It is possible to use these materials as a base and modify them with the GLSL code to work differently.

This thesis aims to make a landscape materials library for one of the WebGL frameworks and explain how the basic modification can be done. Hopefully, after reading this a beginner will gain an insight into the problem and an advanced user will benefit from the library.

The thesis studies the theoretical topics needed to understand the choice of technologies and the final development. At the start, the history of 3D on the web is presented to learn about relevant technologies for rendering on the web. The next topic is JavaScript, the leading language for web development. To make the work with WebGL easier, it is convenient to explore the JavaScript frameworks for 3D graphics. To understand how the materials of 3D objects are evaluated, it is needed to dive into shaders and the language they are written in, called GLSL. The next important topic studies the fundamentals of computer-generated noise as the elemental resource for creating new naturally looking patterns. Last but not least, the FLOSS licenses are analyzed. With all this knowledge, one should be able to understand the final chapter, which describes how the development of modified material is done.

1. History of 3D on the web

3D graphics on the web seems like something new but the opposite is true. It is nearly as old as the World Wide Web. The first mention of 3D on the web is dated to 1994 when David Raggett presented the concept of VRML at the Internet Society conference in Prague. (1) A year later VRML was introduced as the first web-based 3D format and in 1997 was certified by ISO as an international standard ISO/IEC 14772-1:1997. In 2001, VRML was superseded by X3D, which was encoded in XML. Both standards were developed by the Web3D consortium. (2) To display these formats on the web, it was needed to have a plugin installed for a web browser. One of the most installed plugins was the Adobe Flash Player. The boom of the Flash Player led to the development of a new API called Stage3D, which brought GPU acceleration.

With the evolution of the web itself, the possibilities have widened. In 2006, Mozilla Foundation employee, Vladimir Vukićević experimented with the HTML canvas element to display 3D graphics. He came up with the Canvas 3D prototype, which took over Khronos Group in early 2009. They started the WebGL Working Group with initial members such as Apple, Google, Mozilla and Opera. In 2011 this group introduced a WebGL 1.0 Specification. Thanks to binding to OpenGL ES, WebGL enabled hardware accelerated 3D graphics. The most significant benefit was the support in major web browsers without plugins. (3)

WebGL revolutionized graphic rendering on the web. Even X3DOM, which is a newer version of X3D, uses WebGL to render graphics on the web. But since its release in 2011, new native GPU APIs, such as Direct3D 12, Vulkan or Metal, have been introduced. (4) These APIs use a more modern approach to graphics computation, which leads to better performance. The Khronos Group probably will not update OpenGL and WebGL with this approach since they developed Vulkan. Luckily W3C decided to develop a new API called WebGPU. This API is designed from the ground as a standalone connection between GPU and web. (5)

The last paragraph is rising the question of whether developing a library that works with WebGL is still beneficial. The answer is yes. The WebGPU is still in development and its specification is marked as "Working Draft". (5) Moreover web browser support of this new API is under 30 %. (6) Full transition to WebGPU will take some time and the library WebGL implementation may serve as a foundation for WebGPU version.

2. JavaScript

As a JavaScript framework will be used for the completion of the practical part of this work it will be lightly presented in this chapter.

2.1 Overview

JavaScript is a programming language known as the scripting language for web pages. It is also used in desktop applications like Adobe Acrobat or on the server side in frameworks like Node.js and Apache CouchDB. (7)

According to StackOverflow (8), JavaScript has been the most commonly used programming language for eleven years in a row. In 2023, 63.61 % of all respondents marked JavaScript as a programming language, that they used extensively in the past year or want to use in the next one. Among professional developers, it was even as high as 65.82 %.

JavaScript can be divided into two parts. The first is the core language and the second are Web APIs. Core language is standardized by the ECMA TC39 committee. Because of that JavaScript is often called ECMAScript which is the name for the standard. Web APIs are used for interacting with various components of web pages or applications. (9)

JavaScript supports an object-oriented programming style. This style is based on declaring objects that represent particular elements of the system. One specific type of object is called a class. For example, if the system was for an animal shelter, there would be animals like cats, dogs, hamsters etc. Each dog can have common attributes like name, breed or gender. Additionally, they can do some actions like going for a walk. That means each dog is an object and a dog in general is a class. With this approach, it is possible to use inheritance. This means one class can inherit attributes and methods (actions) from another class. Thanks to this, it is possible to reuse code for multiple classes. Returning to the example, all shelter animals have some common attributes and methods but some may differ. This results in a common class animal from which other classes inherit common properties and add some of their own. This can be imagined as extending the common class. During the development of a library that extends the framework, can be the inheritance very useful as it will be seen later.

2.2 History of JavaScript

The initial concept of the World Wide Web has been used among physicists in the first few years since it was developed at CERN by Tim Berners-Lee between 1989 and 1991. A few years later, Marc Andreessen and Eric Bina developed a web client named Mosaic at the

University of Illinois. Mosaic was easy to use and had a graphic user interface. Thanks to that it popularized the World Wide Web to the general public and defined web browser software category. The spreading of Mosaic led to increasing commercial interest in web browsers. The creators of Mosaic co-founded, with Jim Clark, Netscape Communications Corporation to develop their very own web browser, that would replace Mosaic as the world's most popular browser. They achieved this goal with a browser named Netscape Navigator by early 1995. This allowed them to make the next big step. The original World Wide Web concept was built on static pages using HTML but there was interest in making the web more interactive for the end user with the help of scripting languages. Netscape hired Brendan Eich for this purpose. Initially, they wanted to implement languages such as Scheme, Perl or Python, but later Netscape agreed with Sun Microsystems to implement Java language to Netscape 2. The decision to implement Java was primarily made due to the business interests. Java was not a suitable language for the end user, since it was not beginner-friendly. This created a need for an easy-to-use language that would help the user to work with bigger components made with Java. The idea of a big language for professionals and a little language for end users was not anything new. Another example of little languages can be Visual Basic or AppleScript. (10)

The code name for the new language had been chosen Mocha, with a vision of changing it to JavaScript later. The only initial requirement was that Mocha has to look like Java. Among Netscape management, there was a concern about Netscape resources and ability to create scripting language, that would be ready for the Netscape 2 beta release in September 1995. These doubts were dispelled when Brendan Eich made the first Mocha prototype in ten continuous days in May 1995. The prototype was implemented in the pre-alpha version of Netscape 2 and presented to Netscape engineers, who were pleased with the result. JavaScript was officially announced on December 4, 1995, in a press release, by Netscape Communications Corporation and Sun Microsystems. The announcement was aimed at making a strong brand that connects Java and JavaScript even though their technical designs were only slightly similar. (10)

Around the same time, Microsoft announced its intention to make Visual Basic a standard for web applications with its Visual Basic Script language. (11) On May 29, 1996, Microsoft introduced Internet Explorer 3.0 with the support of Visual Basic Script and their reversed-engineered implementation of JavaScript called JScript. Due to these announcements, Netscape and Sun feared that Microsoft would dominate the web browser industry and there would be a preference of the Visual Basic Script. They decided that they needed to standardize JavaScript as soon as possible with Microsoft's participation. Since Sun was already a member of ECMA, they decided to make this standard under ECMA's umbrella. In September 1996 ECMA approved the request and authorized TC39 to organize JavaScript standardization. In the same year's November, representatives of Netscape and Microsoft agreed on the necessity and fundamental requirements of JavaScript standardization at the first TC39 meeting. Later, the first standard version was released under code ECMA-262 with the name ECMAScript. (10)

3. JavaScript libraries for 3D

To work with WebGL, one needs to have skills and be knowledgeable in various industry segments like 3D graphics, JavaScript, GLSL and algorithms that define how 3D objects would look, which are called shaders. It is very challenging because the developer has to code every small step by themselves. This can be overcome with the use of libraries that handle fundamental processes and serve as controllers of the WebGL.

There can be found several libraries that work with WebGL. Each of them has some positives and negatives and may offer different features. It is convenient to do basic research about them before starting the development.

3.1 Babylon.js

Babylon.js is a library that offers plenty of features out of the box and is ready to use. It has nearly everything that is needed, therefore it is easier to start developing application with it. On the other hand, it comes with larger file sizes of source code and prepared classes and functions are not always exactly what the developer requires.

3.2 Three.js

The core of Three.js works only as a WebGL controller with basic classes and functions. Other features like camera control and file loaders are implemented as add-ons. This approach leads to smaller source code and leaves more room for the developer to decide what is needed. Three.js is the most used library for 3D therefore it will be used for this thesis to target as many potential users as possible.

4. Shaders and GLSL

The rendering pipeline has particular stages that can be programmed. Codes for these stages are written in GLSL. The code itself is called a shader. (12)

Shaders need lots of processing power since they may be executed multiple times depending on their type. For example, a fragment shader is executed for each pixel. Just to render one frame of FullHD (1920×1080), the fragment shader is executed 2 073 600 times. For this reason, shaders are processed on GPU which has multiple microprocessors working in parallel at the same time. These microprocessors are designed to specifically do the mathematical operations used by shaders. Parallel processing brings some restrictions. Each execution of the same shader has to be independent from others. In other words, it is impossible to influence the computation of a shader with different execution of the shader. (13)

The type of shader depends on the stage when it is executed.

- Vertex Shaders
- Tessellation Control and Evaluation Shaders
- Geometry Shaders
- Fragment Shaders
- Compute Shaders

For the purpose of this thesis, only vertex and fragment shaders are explored further.

4.1 Vertex Shader

A vertex shader handles the processing of individual vertices. It gets vertex attribute data as input, transforms them and generates a single vertex as output. Since the vertex shader works with vertices, the geometry of the object can be deformed or moved. An example of vertex shader usage can be displacement when vertex shader gets a displacement map and deforms the geometry according to it. In each vertex shader, the position of the vertex should be transformed from world to camera space, since the rest of the rendering pipeline works in camera space.

The output of vertex shader is usually the same while using the same input, and thanks to that it can be optimized with the use of indexed vertices. Each vertex with the same attribute values gets the same index. When it is detected that the vertex shader was already executed for a certain index, the previously computed output will be used instead of a new execution of the shader. (14) A Great example of indexing can be a simple rectangle. A rectangle has four vertices, but in 3D graphics, everything is made of triangles, therefore a rectangle in 3D graphics has six vertices. Out of these six vertices, two pairs of vertices are essentially the same, therefore it is convenient to mark them with the same index and invoke the vertex

shader only once for each index instead of each vertex.

4.2 Fragment Shader

A fragment shader gets fragments as an input and calculates a set of colors and a depth value as an output. Simply said, it dictates how the fragments should look. Fragments are generated in the preceding stage of the rendering pipeline, called Rasterisation. Each fragment represents a pixel of rendered objects. (15)

4.3 GLSL variable qualifiers

GLSL provides several variable qualifiers. They define which shader uses the variable and to what is the variable bound.

Uniforms are global shader variables. They are constant for the entire draw call. That means they have the same value for each invocation of shader within a particular rendering call. Uniforms can be used in vertex and fragment shaders. Usually, the values of uniforms are defined by users.

Attributes are variables, that are bound to each vertex of geometry. Binding to vertices implies that attributes are used only in vertex shader. They can have different values for each invocation of the vertex shader. They work as vertex shader input and are read-only during the whole scope of the vertex shader. The values of attributes are usually position, UV coordinates and normals of vertices.

Sometimes it is needed to use attributes in a fragment shader, but from their definition it is impossible, therefore varyings exist. They work and behave differently depending on the shader stage. Their value is computed and written to them in the vertex shader and then used as the output of the vertex shader. In fragment shader, it is the complete opposite. They are the input of fragment shader and in the whole scope are read-only. Thanks to that varyings can be used for sending values from vertex shader to fragment shader. Since the number of vertices is different than the number of fragments and fragments can be between vertices, varyings have to be recalculated. It is done during rasterisation. WebGL gets vertices that form a triangle in which the fragment is located. The value of varying for fragment is calculated as an interpolation between values of varyings of vertices based on the distance between fragment and vertices.

Attribute and varying are equivalents to in (input) and out (output) qualifiers depending on which type of shader are they used in. Because of that, they were deprecated in version 1.3 of GLSL and version 1.4 of GLSL removed, however, WebGL supports GLSL version 1.00, therefore it is safe to use. Moreover, they are more meaningful.

4.4 GLSL preprocessor directives

Since GLSL is similar to the C language, it has a lot of the same features. Preprocessor directives are one of them. They are not parts of the final code, but they influence it. When the GLSL code is compiled, it is parsed with a preprocessor. With directives, it is possible to tell the preprocessor which part of the code can be omitted or use macros for recurrent constants. Since preprocessor directives are evaluated during compilation, they cannot operate with runtime variables and any change that depends on them, results in the need for recompilation.

To create macros serves the directive `#define`. It can be useful for defining constants as π with lots of decimals instead of writing them over and over through the code.

Another useful directive is `#ifdef` which pairs with `#else` and `#endif`. Negation can be done with `#ifndef`. This directive creates a conditional block of code, that is compiled only if the given macro is defined or is not falsy. Sometimes could be needed to compare the value with a constant. For this purpose has to be used directive `#if`.

```
#define DISTANCE 42

#ifdef DISTANCE

  // DISTANCE is defined => this block of glsl will be compiled

#else

  // DISTANCE is defined => this block of code will not be compiled

#endif
```

To reuse part of code in multiple different shaders can be used directive `#include`. The preprocessor copies the part of the code to the place of the directive during compilation. This can be useful, especially for generic functions like sampling and evaluating textures.

GLSL has more preprocessor directives, that could be useful but these will not be mentioned in this thesis.

5. Noise

The world is full of diverse patterns; ocean waves, structure of wood, marble or lichen on the rocks. To achieve this randomness of patterns in computer graphics noise algorithms are used. One of the most famous noise algorithms is the Perlin noise. This algorithm was created by Ken Perlin in the 1980s when he was creating more realistic textures for the movie Tron.

5.1 Perlin Noise

As Perlin (16) states, Perlin noise has three elemental properties.

- Statistical invariance under rotation - statistical character is independent of domain rotation
- A narrow bandpass limit in frequency - its features are similar in size
- Statistical invariance under translation - statistical character is independent of domain translation

The Perlin noise algorithm was originally designed as 3D noise, which means it takes a three-dimensional vector as an input. Its fundamental logic can be applied in fewer or more dimensional spaces. Noise generation is done in two main steps.

1. *"Consider the set of all points in space whose x, y, and z coordinates are all integer valued. We call this set the integer lattice. Associate with each point in the integer lattice a pseudorandom value and x, y, and z gradient values. More precisely, map each ordered sequence of three integers into an uncorrelated ordered sequence of four real numbers:* $[a,b,c,d] = H([x,y,z])$*, where [a,b,c,d] define a linear equation with gradient* $[a,b,c]$ *and value d at* $[x,y,z]$*.* $H()$ *is best implemented as a hash function.*
2. *If* $[x,y,z]$ *is on the integer lattice, we define* $Noise([x,y,z]) = d_{[x,y,z]}$*. If* $[x,y,z]$ *is not on the integer lattice we compute a smooth (eg. cubic polynomial) interpolation between lattice equation coefficients, applied first in x (along lattice edges), then in y (within lattice z-faces), then in z. We then evaluate this interpolated linear equation at [x,y,z]."*

(16, p. 289)

In other words, this means

1. Generating pseudorandom real number value for each point of integer lattice.
2. Use these values for points on the integer lattice as noise values and for points that are not on the integer lattice make smooth interpolation between the closest points of the integer lattice.

5.2 Simplex Noise

The original Perlin noise algorithm has some deficiencies, therefore Ken Perlin came up with an improved algorithm named Simplex Noise. The first shortcoming is in the cubic interpolation function that is used for smoothing. Its second derivate is not a zero in edge cases (around integer lattice points), which leads to sharp edges. The second deficiency is in the distribution of gradients. Since gradients are calculated as a linear interpolation of eight points of a cube, they are elongated in a diagonal direction. This causes a clumping effect in regions where gradients are almost axis-aligned and close together causing anomaly high values. These deficiencies were solved by replacing the interpolation function, skewing cubic space and calculating gradients to edges instead of points. These modifications led to visual improvements and performance efficiency. (17) Simplex noise was patented, but the patent had already expired therefore it is free to use.

5.3 Fractal Brownian Motion

As was already described, Perlin and Simplex noises have features similar in size. This is not visually appealing in some cases. To solve this the fractal Brownian Motion or simply fractal noise is used. This technique sums multiple iterations of the noise together. Iterations are called octaves. Each consecutive octave should have increased frequency (size of features) and decreased amplitude (maximum and minimum value). (13)

Each additional octave adds more granularity to the final noise which is more visually appealing. On the other hand, each octave means additional computation of noise, therefore number of octaves is usually a compromise between the visual look and the computation efficiency. In the figure 5.1 the effect of different numbers of octaves can be observed. The difference between one and three octaves is noticeable at first sight but the difference between the three and the six is not that prominent.

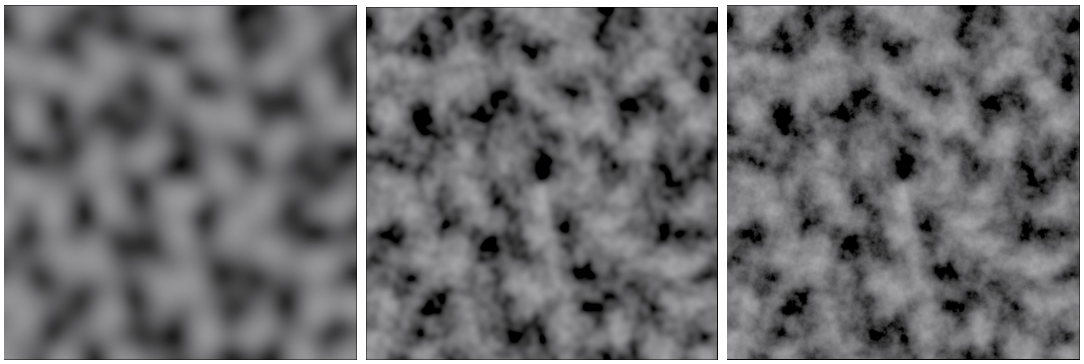

Figure 5.1: Simplex noise with one, three and six octaves

6. FLOSS - an approach to licensing

An outcome of this work's practical part is a piece of code that needs to be licensed. It will be released under one of the FLOSS licenses. Therefore is need to look into the possibilities of such licenses.

FLOSS is neutral ground between two political camps, free software movement and open source. Both groups promote the idea of free software but each group has a different point of view on the topic. While the free software movement sees free software as freedom for users, open source sees only practical benefits as will be explained further later. In practice, it is very similar but free software movement is more free than open source. (18)

FLOSS is an abbreviation for Free/Libre and Open Source Software. Sometimes it is used term FOSS which has the same meaning but can be misleading because of the omitted word libre. Free in this context stands for freedom and not the price. For this reason, it is advised not to omit the Spanish or French word libre which means free in the sense of freedom. (18)

The open source licenses tend to be more restrictive because they are defined more as conditions rather than what users are free to do. For example, some prohibit making modified versions of the software. On the other hand free software uses copyleft that ensures freedom of software and all modified and extended versions. It is done by copyrighting software and then adding the distribution terms that allow the distribution, use, modification and development of derivatives to everyone, but only if the distribution terms are unchanged. (19)

Choosing the right license can be overwhelming at first glance because of the number of license types, but in the end, only a few are widely used. When choosing a license it is important to think about the compatibility with other licenses in case of combining software to a larger work.

The rest of this chapter is a brief summary of the most used licenses for similar purposes as the outcome of the practical part of this work. When choosing a license, everyone should read the original wording of the license.

6.1 Public Domain

Public Domain is not an actual license. It is a term for an intellectual property, that is not protected by copyright, trademark or patent laws. Public Domain means that the property cannot be owned by the individual but public itself. Since it is owned by the public, anyone can use it without the need for obtaining permission. Intellectual property can be released as public domain by its author by not adding any license when releasing or it can become public domain by the copyright expiration. (20)

6.2 GNU General Public License v3

GNU General Public License is written as a copyleft license, which means that the software licensed with this license is free and stays free no matter who changes it or distributes it. GNU General Public License also protects against legal regulations and technological processes that could threaten copyleft. Some companies use software with this license on their devices, which can lead to situations when users cannot modify the software. Which is against the free software idea and breaks the license rule. GNU General Public License solves this by requiring distributors to provide users with anything necessary to modify and use modified software. This license provides protection against patent threats. Everyone who covers their software with this license is obligated to provide every user with patent licenses required to use their rights given by the license. Whenever someone uses a patent suit against another user, their license will be terminated. (21)

6.3 MIT license

MIT license is an ambiguous term because it is used for two similar licenses, Expat and X11. (22) The difference is an extra paragraph in X11 that states: *Except as contained in this notice, the name of the X Consortium shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Software without prior written authorization from the X Consortium.* (23). Both licenses grant permission to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software without limitation. They provide software "AS IS", which means without any warranty or liability. The license text has to be in all copies or substantial portions of the Software.(23)

6.4 Apache license 2.0

Apache license is very similar to the MIT license but adds protection against patent litigations. Each user is granted with patent license from every contributor. If the user uses the patent license to institute patent litigation, their license will be terminated. (24)

6.5 Chosen license

Since Three.js is licensed under MIT, specifically Expat, license, this library will be licensed under this license as well. The Expat license is suitable and sufficient for this library. Moreover using the same license ensures license compatibility.

7. Preliminary research

Preliminary research was conducted in the form of a questionnaire. It consisted of three sections. The first section presented the topic of this thesis and asked about its usefulness. The second section was branched in compliance with the previous question. Positive answer led to a branch with questions about implementation and suggestions for other shader variants. Negative tried to find out what was the problem. The last section was for other suggestions and getting email addresses for notification about development results.

Responses were collected between the 20th of September and the 6th of October 2023. Unfortunately, not many responses were collected as the topic is very particular. All responses however agreed on the need for such a library and suggested procedurally generated materials, tire tracks on highways, and moisture & temperature based shading for different biomes.

There were also suggestions, that were already mentioned in the initial idea like blending more materials (e.g. terrain with partial snow and partial grass) and avoiding texture repetition. One respondent also suggested a highlight/outline that is more suitable for the library with materials for objects than landscapes. The majority of responders agreed on wrapping shader code in material class as this type of implementation is the easiest to use.

Suggestions for the library from a general point of view were such as:

- making examples with options;
- sufficiently detailed documentation;
- suitable license of the source code so that people can mix and update the shaders.

8. Development of material modifying library for Three.js

The library was designed as a set of new Three.js materials. This approach was chosen as the easiest way to use it because users of Three.js are familiar with core materials. An additional positive is that the users do not have to handle GLSL shaders through the ShaderMaterial class which has a different workflow than others.

Creating new material can be done with two approaches. Make whole material from scratch or modify some of the existing materials. The first option ensures control over the whole code and results in independent material classes, that will not break after updating the original. On the other hand, it takes a lot of effort to create a material foundation, moreover, it is like reinventing the wheel since it is already done. The modification of existing classes can introduce problems in compatibility with new versions of the original library. Luckily, the community that releases Three.js tries to not make changes, that would cause incompatibility between versions. Especially not in core kinds of stuff like materials. The positives prevail in this case. As was already stated above, not reinventing the wheel speeds up the implementation of new materials. Another great aspect of modifying existing material is that all new features of the original class, which do not introduce compatibility issues, are automatically implemented in the new one.

All materials in the library were MeshStandardMaterial class extended with new attributes and its shader code was injected with new parts of code specific to each material. MeshStandardMaterial was chosen as the base class because it is physically based rendered material which is modern standard in 3D application. This type of material has more realistic-looking results than older approaches, like the Lambert or Phong lighting model, at the cost of being more computationally expensive.

8.1 Modification of Three.js materials

Modification is done in three basic steps. Firstly extending the javascript material class and secondly injecting its shader code as was stated above. The third step is connecting attributes of the javascript class with GLSL shader code. To successfully modify material is necessary to know the javascript and GLSL code of the original material.

Extending a class means creating a new class, that inherits from the parent class and adds new attributes and methods, thus extending the parent class. Besides adding new properties, it is usually useful to change the existing ones, like changing the default values of attributes or the behavior of methods.

Shader code written in GLSL is stored as a string in WebGL. This code is compiled before the first render. Three.js material classes have a method called `onBeforeCompile`, which is executed before shader compilation, as the name suggests. In this method, it is possible to rewrite the shader code. It is done by using the `replace` method provided by the javascript string. The `replace` method takes two arguments. The first is a part of the original string to be replaced and the second is a new string. Thanks to this it is possible to add new parts, remove old ones or both at the same time. While injecting the code it is needed to choose where the new code should be placed and then choose a substring from this area. This substring has to be unique in the whole original string because the `replace` method replaces only the first occurrence. This behavior can be useful in some cases but usually is not. The original materials in Three.js have GLSL code assembled from smaller part, called chunks, with `#include` directive. Each of these chunks represents part of the material calculation, like the calculation of diffuse colour, reflections or transparency. Thanks to this it is possible to use the directive as the substring that will be replaced because `includes` are always unique and usually the injected code is related to these parts. Sometimes it is needed to separate the new code into chunks and use the `replace` method for each chunk to inject it into different parts of the code. Usually, it is needed to use `replace` at least two times per shader. Once for the chunk that defines uniform, varyings and attributes and once for the main function.

To connect javascript class attributes with GLSL code, two javascript objects are used, `uniforms` and `defines`. Both names correspond with keywords used in GLSL. Each attribute that is needed in GLSL code has to be saved in one of these objects. `Defines` should be used only for attributes that will not be changed too much during runtime. Since `defines` are relevant only during compilation, the material has to be recompiled after changing these. They are usually booleans used for determining which part of code will not be used and can be omitted during compilation. All attributes, that the user might want to change more frequently, should be stored as uniforms.

8.2 Texture repetition (tiling)

One of the most frequent problems that all 3D artists encounter is texture tiling or repetition. Using seamless texture is a great starting point but no matter how good the texture is, it has always some pattern that is visible with more repetition as can be seen in the figure 8.1.

8.2.1 Research

There are several techniques how to solve this problem. Each of them has its positives and negatives. Usually, a combination of several techniques is used.

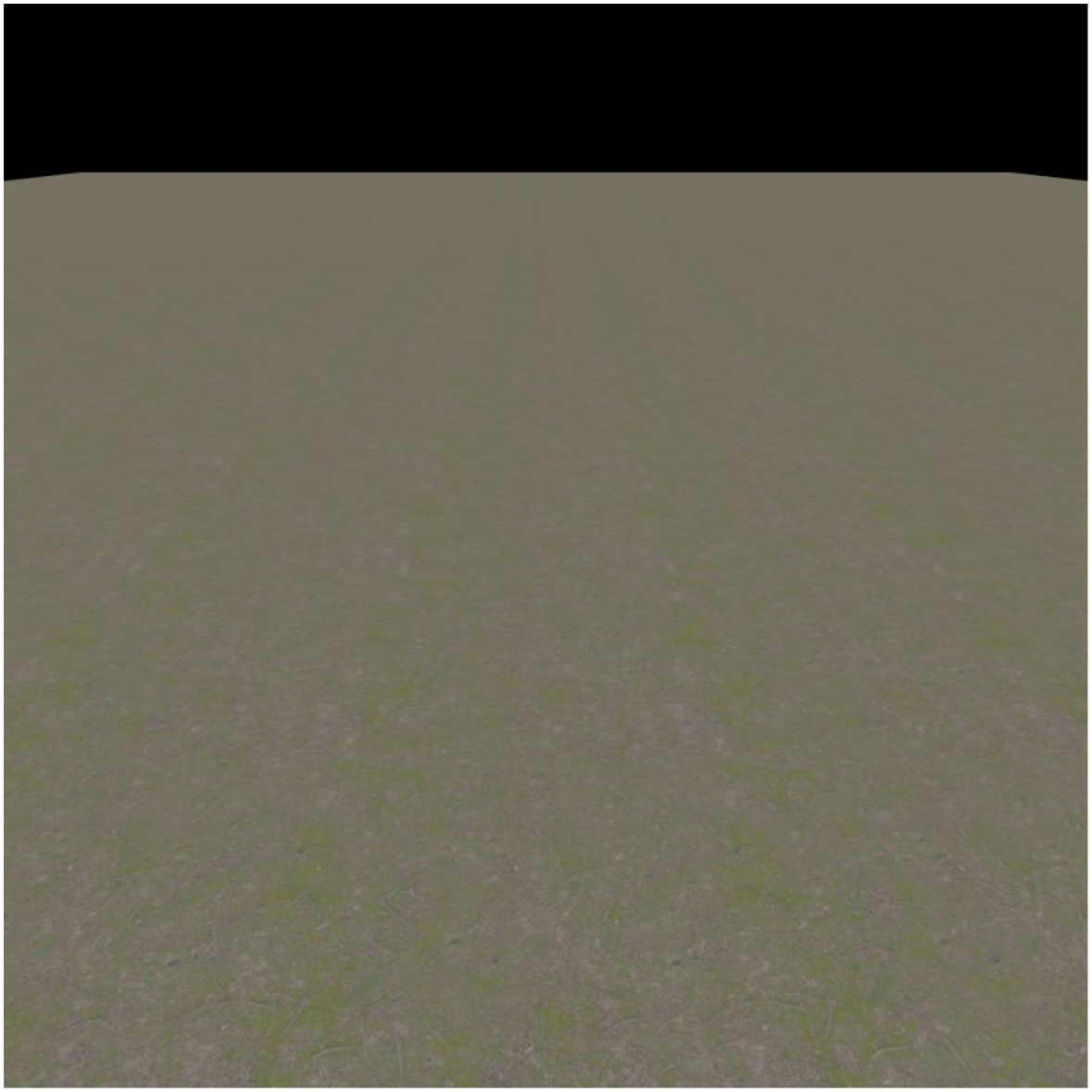

Figure 8.1: Plane with seamless texture

Texture bombing

This method breaks the texture grid by taking multiple texture samples, offsetting and rotating them by pseudorandom numbers and then blending them together. This leads to good results, but some types of textures do not work with this approach well. For example, rotation of pavement texture would lead to weird results. Another problem is performance. Instead of sampling once per texture type (diffuse, normal, roughness, etc.), now the shader has to sample multiple times per type.

To lower the performance impact it is possible to break the texture grid with the Voronoi texture. Each cell of the Voronoi texture represents a pseudorandom offset of UV coordinates. With this offset, the original material texture is sampled. This leads to only two samples (one for Voronoi and one for material texture) per texture type. This helps with performance efficiency but introduces a new problem with blending the texture on the edges of Voronoi cells. Once again the example with pavement, the offset in two adjacent cells would be half of the pavement brick. This would result in seam blending to the middle of a brick.

This technique can be great for some scenarios, but it is not universal for everything.

Micro and macro variation

Instead of breaking the texture grid, this technique adds variations to it. Textures are usually images of a few meters of surface. Because of that, they cannot capture variations of surface on a larger scale. It can be found on every type of surface. Grass, concrete, pavement, and even dirt, all of them are usually worn down by walking people or rain. Random parts of the surface are faded or have different tints of colour.

Micro variation is used to make tint variations of the surface. It uses noise or noise texture as an alpha channel to distinguish where to change the tint of the colour.

Macro variation makes various parts of the surface darker or lighter. It uses grayscale noise or noise texture as a multiplier of surface colour. Macro variation is sometimes used also on the roughness texture, but usually, it is not necessary.

Both variations should be scaled to cover a large portion of the surface, especially when using a noise texture, otherwise the repetition of the variation could be visible. Also, micro variation should have a different noise pattern and scale than macro variation to not overlap each other.

Distance tiling

Distance tiling changes the texture scale depending on the distance to the camera. Farther parts of the terrain are scaled up compared to the closer ones. It lowers the repetition of

texture on farther parts of the surface, where it is most noticeable.

This technique is great for cases, where the camera is near the surface, like a first or third-person camera. For cases, where the camera is very high, like a top-down or airplane camera, this technique is not suitable because the distance between the camera and various parts of the surface is similar, and so is the texture scale. It can be sufficient on its own with good texture or it can be great in addition to other techniques.

8.2.2 Implementation

It was decided to implement material that solves texture repetition by using micro and macro variation in combination with distance tiling. Each of these steps is optional and the user can decide if it is needed or not. The vast majority of these techniques logic is done in fragment shader.

The definition of camera distance varying in the vertex shader is injected after $\#include < common >$ which includes common inputs for shaders. The camera distance calculation is injected after $\#include < project\_vertex >$ which calculates the position of the object in the viewport. For the fragment shader, are used chunks that calculate diffuse colour texture. Inputs of fragment shader are injected after $\#include < map\_pars\_fragment >$. The calculation replaces $\#include < map\_fragment >$, which is used in the original material to sample the diffuse color texture. Since distance tilling changes the behavior of sampling the texture, the new code has to replace the original one instead of adding it like in the previous cases.

Distance tiling has to be done first since it affects other steps. It calculates camera distance in the vertex shader and sends it to the fragment shader. There it samples texture two times. Firstly with base UV coordinates and then with scaled ones to magnify the texture. The magnification factor is a parameter of the material, so it can be chosen by the user as well as near and far distance. These two distances are used to map the camera distance between 0 and 1. Camera distance lower than near distance is always 0 and higher than far distance is always 1. The mapped value is then used to interpolate between two texture samples. In the figure 8.2 can be seen the same plane as in the figure 8.1, but with applied distance tilling. The more distant parts of the plane now have texture magnified by a factor of 2.7. The texture repetition can still be noticed but it is not as prominent as before.

Part of the fragment shader that solves distance tilling:

```
vec4 nearColor = texture2D( map, vMapUv );
vec4 farColor = texture2D( map, vMapUv / farTextureMagnification );
diffuseColor *= mix(nearColor, farColor, smoothstep(nearDistance,
    farDistance, clamp(vCameraDistance, nearDistance, farDistance)));
```

There was an attempt to make the algorithm of distance tiling more efficient by scaling UV coordinates with camera distance. Sampling texture would be done with scaled UV

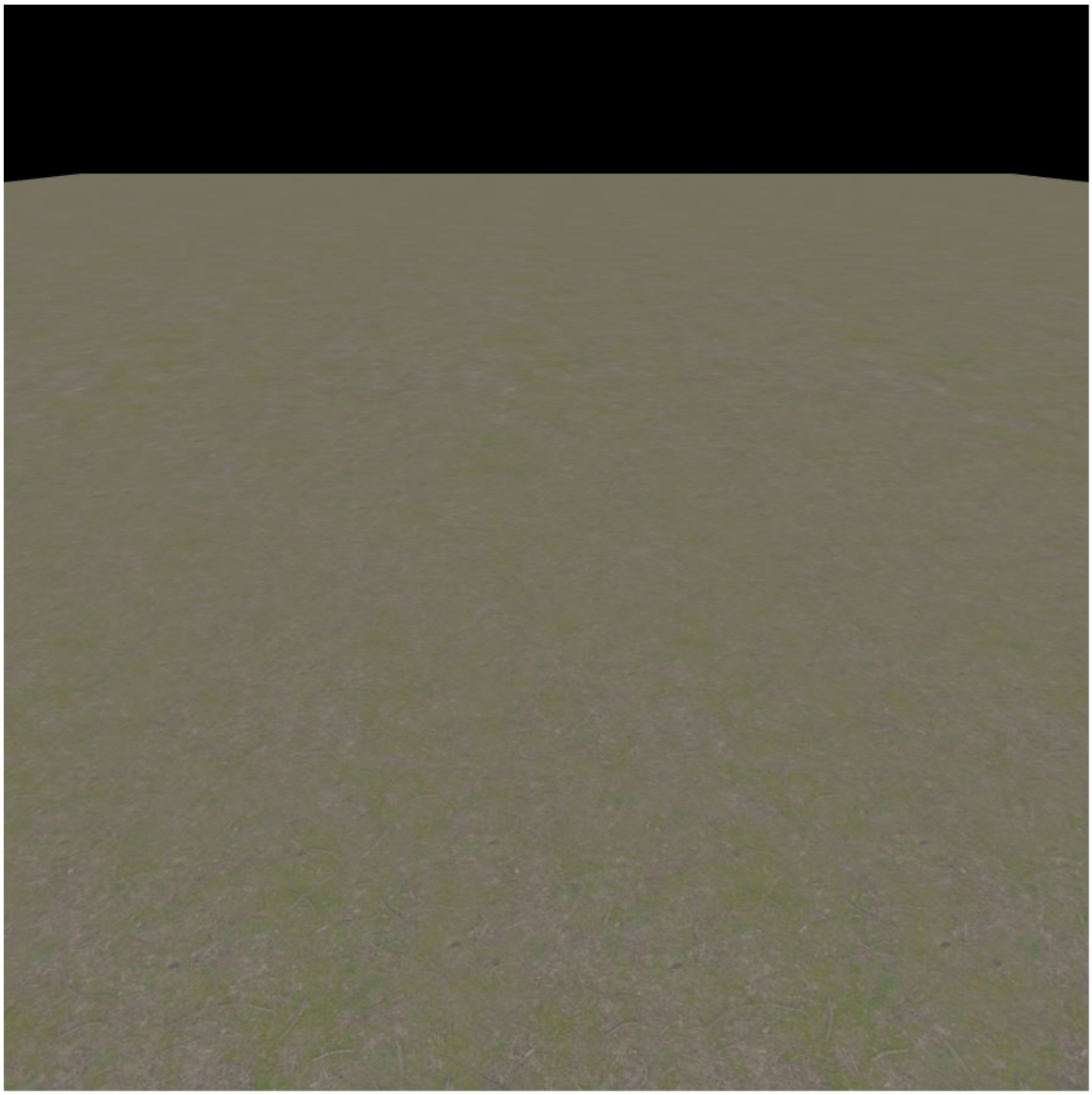

Figure 8.2: Plane with distance tilling

coordinates resulting in one texture sampling instead of two. Unfortunately, this approach turned out to be wrong. Scaling UV coordinates with different factors for different parts of geometry leads to the deformation of the UV map thus sampled texture is deformed as well.

The micro variation samples texture or uses the result of distance tiling and multiplies it with colour to get a varied tint of texture. Then it does linear interpolation between original and tinted textures at the value of fractal Simplex noise. The colour is a parameter that can be changed by the user. The result of micro variation is a plane with different colour regions as can be seen in the figure 8.3.

Part of the fragment shader that solves micro variation:

```
vec4 microVariation = vec4(microColor, 1) * diffuseColor;
float microAlpha = fbm(vPosition.zx + 3652.0, 6, 0.05, 1.0);
diffuseColor = mix(microVariation, diffuseColor, microAlpha);
```

The macro variation takes the value of fractal Simplex noise with multiple octaves to interpolate between contrast and constant 1 to change the contrast of the noise. The result is multiplied by the micro variation result or material texture depending on if the micro variation was used. The contrast is a parameter that can be changed by the user. In the figure 8.4 is shown the result of the macro variation algorithm applied. Where some spots of the texture are lighter because the value of contrast was higher than 1. If the value had been less than 1, the spots would be darker.

Part of the fragment shader that solves micro variation:

```
float macroAlpha = fbm(vPosition.xz + 3423.0, 6, 0.05, 1.0);
macroBlend = mix(1.0, macroContrast, macroAlpha);
```

Both variations use position coordinates as inputs for the simplex noise algorithm. These coordinates are multiplied by a constant, which is different for each variation, to ensure different noise patterns. In the future, this constant can be changed to a user-defined variable to serve as a random seed.

Usage of all methods applied together can be seen in figure 8.5.

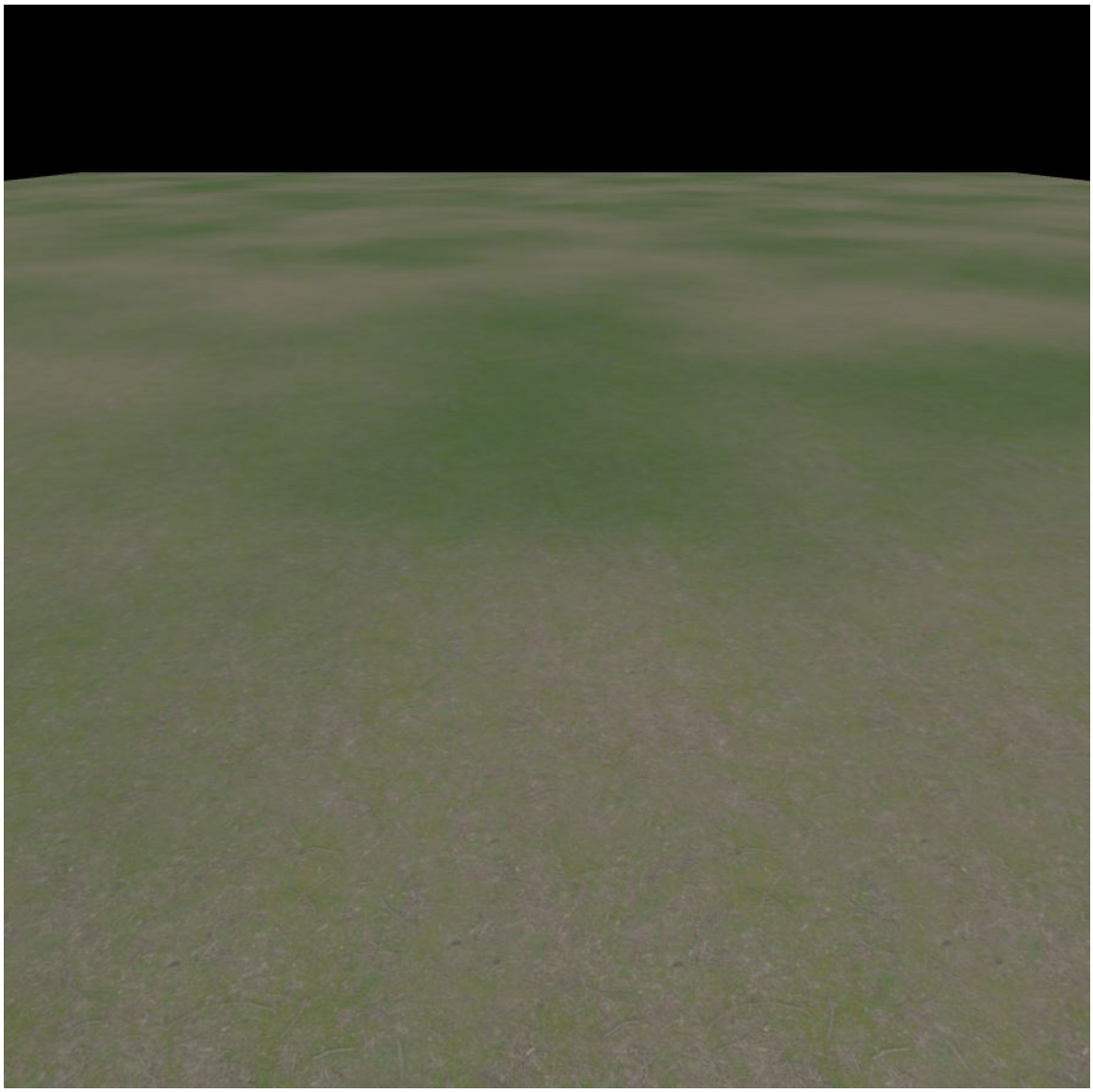

Figure 8.3: Plane with micro variation

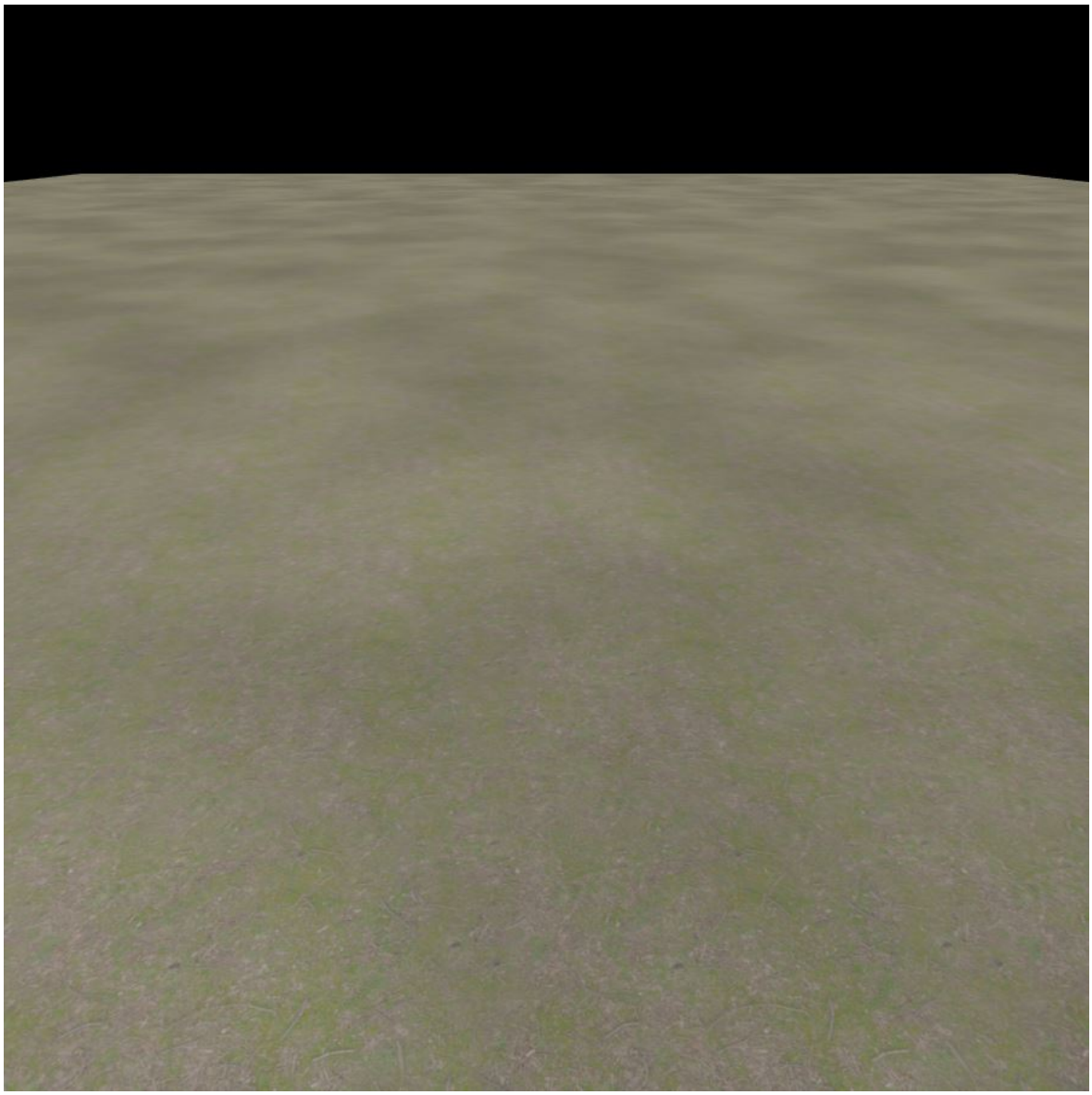

Figure 8.4: Plane with macro variation

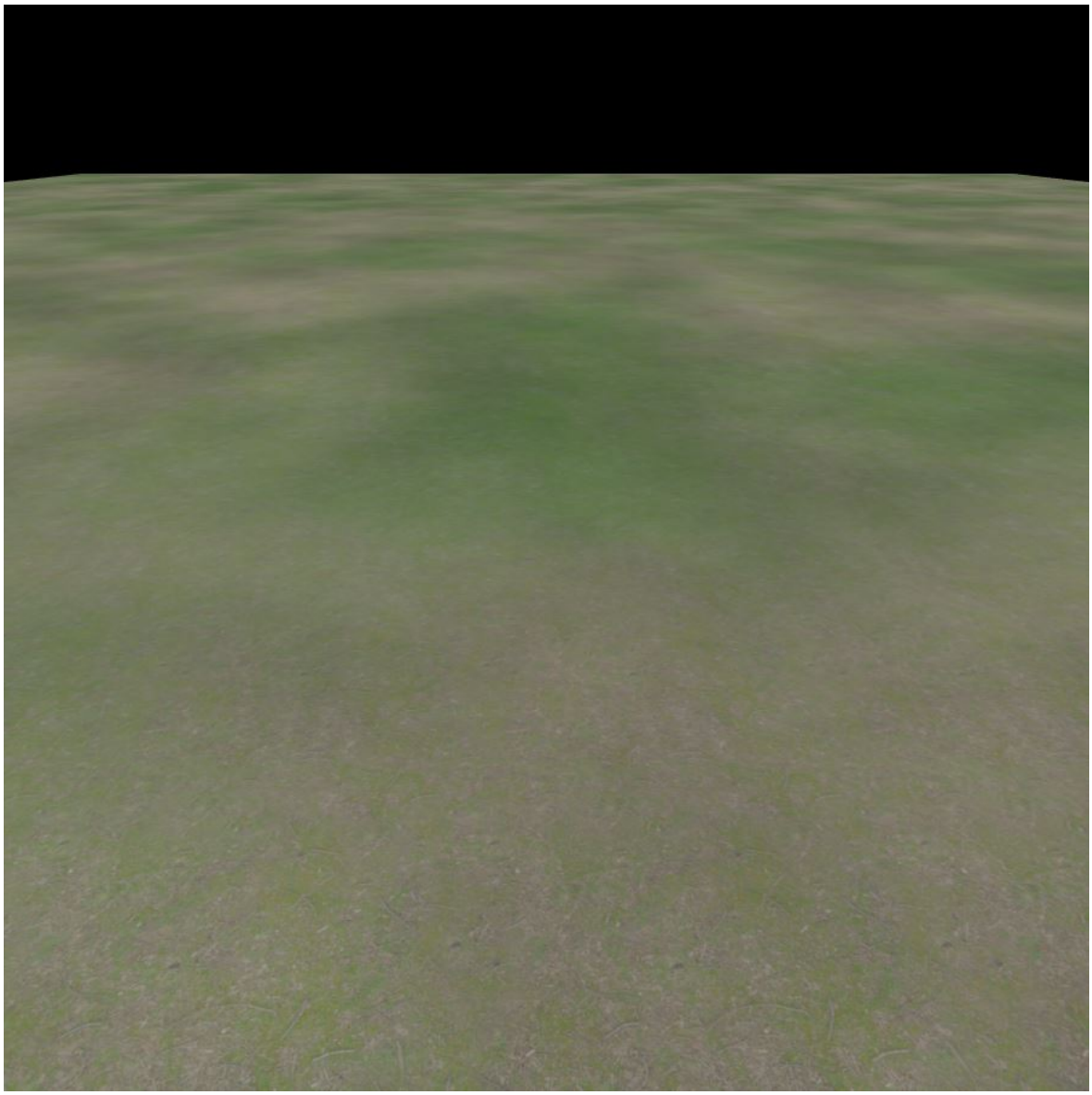

Figure 8.5: Plane with all methods

Conlusion

The goal of this bachelor thesis was to implement the library of materials, that would help users with the creation of landscapes in the web environment. This goal arose from the problem of texture repetition in general 3D space and then specifically in a 3D web environment. The goal is partially fulfilled since the initial problem of the texture repetition was solved but in the library, any of the other material proposed in the answers to the questionnaire was not implemented.

The library was made for the Three.js framework as it is the most used framework for working with 3D graphics on the web. From all of the Three.js materials, the MeshStandardMaterial was chosen as the base material that will be modified. It was modified by creating a new class that inherited properties from the MeshStandardMaterial and new ones were added. On top of that, the GLSL code was altered by adding new algorithms and rewriting some of the old ones. These steps are universal when a material is being modified. The specific things for each material are which properties are added and the GLSL code algorithms. Specifically, the created material was based on distance tilling and macro and micro variation techniques.

Even though the initial problem was solved, future development is still needed. The most obvious thing that needs to be developed is other materials that were mentioned in the questionnaire. For the created material that hides texture repetition, it would be useful to add more options for noise generation. For example, the following could be added: algorithms for different noises or more dimensional noises, an option for changing the number of octaves and for the already mentioned seed. Some users could also possibly want the option to use pre-generated noise texture instead of generating noise during runtime. This option would boost performance when using more complex noise with multiple dimensions and octaves. On the other hand, it could introduce one or a combination of problems like a noticeable repetition of the noise pattern, low resolution of the noise pattern or high memory size depending on how the texture would be used and the texture itself. In other words, the material is usable but still can be improved.

Bibliography

1. REGGETT, David.
   Extending WWW to support Platform Independent Virtual Reality [Online].
   [visited on 2023-12-07].
   Available from: https://www.w3.org/People/Raggett/vrml/vrml.html.
2. WEB3D CONSORTIUM.
   X3D & VRML, The Most Widely Used 3D Formats [Online]. [visited on 2023-12-07].
   Available from: https://www.web3d.org/x3d-vrml-most-widely-used-3d-formats.
3. THE KHRONOS GROUP INC.
   Khronos Releases Final WebGL 1.0 Specification [Online].
   The Khronos Group Inc. [visited on 2023-12-08].
   Available from: https://www.khronos.org/news/press/khronos-releases-final-webgl-1.0-specification.
4. MOZILLA FOUNDATION. *WebGPU API* [Online]. [visited on 2023-12-08]. Available from: https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API.
5. THE WORLD WIDE WEB CONSORTIUM (W3C). *WebGPU* [Online].
   2023-12. [visited on 2023-12-08].
   Available from: https://www.w3.org/TR/2023/WD-webgpu-20231206/.
6. CAN I USE. *WebGPU* [online]. Can I use. [visited on 2023-12-08].
   Available from: https://caniuse.com/?search=webgpu.
7. MOZILLA FOUNDATION. *JavaScript technologies overview* [Online].
   [visited on 2023-12-08]. Available from: https://developer.mozilla.org/en-US/docs/Web/JavaScript/JavaScript_technologies_overview.
8. STACKOVERFLOW.
   Stack Overflow Developer Survey 2023: Most popular technologies [online].
   Stack Overflow, 2023. [visited on 2024-05-03]. Available from: https://survey.stackoverflow.co/2023/#most-popular-technologies-language-prof.
9. MOZILLA FOUNDATION. *JavaScript* [Online]. [visited on 2023-12-08].
   Available from: https://developer.mozilla.org/en-US/docs/Web/JavaScript.
10. WIRFS-BROCK, Allen and EICH, Brendan. JavaScript: the first 20 years.
    Proceedings of the ACM on Programming Languages.
    2020-06, vol. 4, no. HOPL, pp. 1–189. ISSN 2475-1421.
    Available from DOI: 10.1145/3386327.
11. WINGFIELD, Nick. Microsoft storms the Web. *InfoWorld*. 1995, vol. 17, no. 50, p. 24.
    Available also from:
    https://books.google.cz/books?id=QjgEAAAAMBAJ&lpg=PP1&dq=Inforworld%20Dec%2011%2C%201995&pg=PP3&redir_esc=y#v=onepage&q&f=false.

12. THE KHRONOS GROUP INC. *Shader* [Online].
OpenGL Wiki, 2019-10. [visited on 2023-12-07].
Available from: `https://www.khronos.org/opengl/wiki/Shader`.

13. GONZALEZ, Patricio and LOWE, Jen. *The Book of Shaders* [Online]. 2015.
Available also from: `https://thebookofshaders.com/`.

14. THE KHRONOS GROUP INC. *Vertex Shader* [Online].
OpenGL Wiki, 2017-11. [visited on 2023-12-07].
Available from: `https://www.khronos.org/opengl/wiki/Vertex_Shader`.

15. THE KHRONOS GROUP INC. *Fragment Shader* [Online].
OpenGL Wiki, 2020-11. [visited on 2023-12-07].
Available from: `https://www.khronos.org/opengl/wiki/Fragment_Shader`.

16. PERLIN, Ken. An Image Synthesizer. *ACM Siggraph Computer Graphics*.
1985, vol. 19, no. 3, pp. 287–296. Available from DOI: `10.1145/280811.280986`.

17. PERLIN, Ken. Improving Noise. *ACM Transactions on Graphics*.
2002, vol. 21, no. 3, pp. 681–682. Available from DOI: `10.1145/566570.566636`.

18. STALLMAN, Richard. *FLOSS and FOSS* [Online]. The GNU Operating System and the Free Software Movement, 2021-09. [visited on 2023-12-06].
Available from: `https://www.gnu.org/philosophy/floss-and-foss.html`.

19. STALLMAN, Richard. *Why Open Source Misses the Point of Free Software* [Online]. The GNU Operating System and the Free Software Movement, 2023-10. [visited on 2023-12-06]. Available from:
`https://www.gnu.org/philosophy/open-source-misses-the-point.html`.

20. STIM, Rich. *Welcome to the Public Domain* [Online].
Stanford Copyright and Fair Use Center. [visited on 2023-12-09]. Available from:
`https://fairuse.stanford.edu/overview/public-domain/welcome/`.

21. SMITH, Brett. *A Quick Guide to GPLv3* [Online]. The GNU Operating System and the Free Software Movement, 2022-01. [visited on 2023-12-09].
Available from: `https://www.gnu.org/licenses/quick-guide-gplv3.html`.

22. FREE SOFTWARE FOUNDATION'S LICENSING AND COMPLIANCE LAB.
Various Licenses and Comments about Them [Online]. The GNU Operating System and the Free Software Movement, 2023-10. [visited on 2023-12-09].
Available from: `https://www.gnu.org/licenses/license-list.html.en#Expat`.

23. XCONSORTIUM. *Other Copyrights* [Online]. 1996-10. [visited on 2023-12-09].
Available from: `https://www.xfree86.org/3.3.6/COPYRIGHT2.html`.

24. THE APACHE SOFTWARE FOUNDATION.
APACHE LICENSE, VERSION 2.0 [Online].
The Apache Software Foundation, 2004-01. [visited on 2023-12-09].
Available from: `https://www.apache.org/licenses/LICENSE-2.0`.

Appendices

A. Source codes of computational procedures

All of the source codes created for the library can be found in the repository on GitHub. `https://github.com/Riwox/threejs-landscape-materials`